Problem 34 Solution

Erik Johnson

November 13, 2017

NOT APPROVED BY ANDY RUINA

Problem

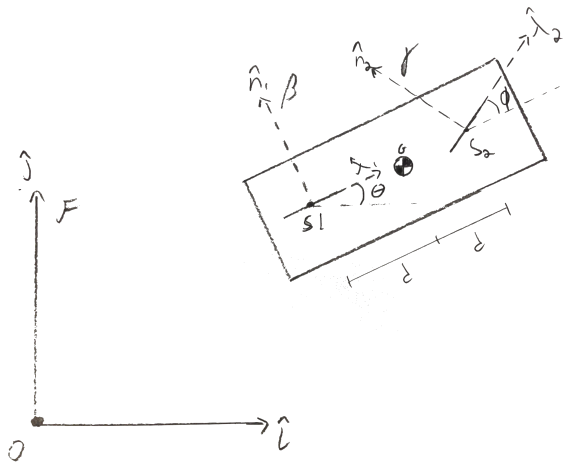
A rigid object (m, I_G) moves on the plane with no gravity or other forcing. It has on it two skates that are not parallel, and neither one is on the normal line of the other.

(a) Find the motion by writing unconstrained Lagrange Equations. Then calculate generalized forces associated with the constraints. Then supplement the Lagrange equations with constraint equations (two times over: write the skate velocity constraint and differentiate it). These are DAEs coming from Lagrange equations. Then solve these equations numerically using example parameters of your choosing. [Hints: 1) Your DAEs only enforce the constraint at the accelerationlevel. So, in your numerical solution the initial velocities need to be consistent with the kinematic constraints. 2) Your DAEs should be the same as you would get using Newton-Euler]

(b) The solution above can also be found by simple elementary means. If you don't see this, make an animation of the solution above. Compare the result.

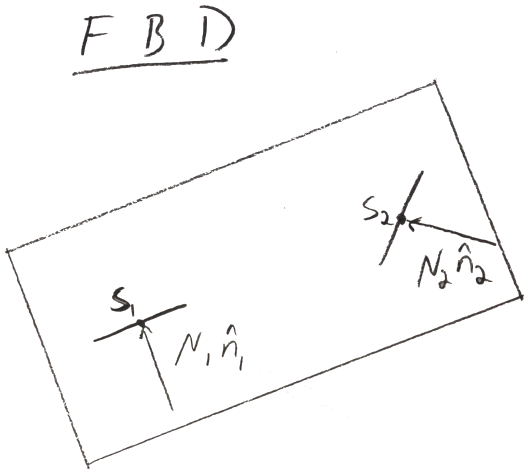
Solution

First, we can set up the problem in a diagram. Here, I have fixed the skate S_2 to always point along the $\hat{\lambda}_1$ direction. The angle θ is measured from $\hat{i}$ and $\hat{\lambda}_1$. θ will be used to describe the rotational motion of the cart with respect to the inertial frame, F. The second frame, γ, is fixed along the second skate, S_1. The angle ϕ is measured from λ_1 to λ_2. This angle is fixed. We will assume that the COM is at the center of the cart and that d is the distance from the COM and each skate.



$$\begin{bmatrix} \hat{\lambda}_1 \\ \hat{n}_1 \end{bmatrix} = \begin{bmatrix} cos(\theta) & sin(\theta) \\ -sin(\theta) & cos(\theta) \end{bmatrix} \begin{bmatrix} \hat{i} \\ \hat{j} \end{bmatrix}$$

$$\begin{bmatrix} \hat{\lambda}_2 \\ \hat{n}_2 \end{bmatrix} = \begin{bmatrix} cos(\theta) & sin(\theta) \\ -sin(\theta) & cos(\theta) \end{bmatrix} \begin{bmatrix} cos(\phi) & sin(\phi) \\ -sin(\phi) & cos(\phi) \end{bmatrix} \begin{bmatrix} \hat{i} \\ \hat{j} \end{bmatrix}$$



(a)

First, we must describe the motion of the cart using unconstrained Lagrange equations. Upon inspection of the diagram, we can describe the cart's motion using x_G, y_G, and θ. Looking at the FBD above, we also know that we have two normal forces, N_1 and N_2 that we must use to constrain the motion of the cart. Therefore we need five sets of DAE equations to form our A and b matrices:

- LMB (x2) $\rightarrow$ (2x) Lagrange
- AMB (x1) $\rightarrow$ (1x) Lagrange
- Constraint (x2) $\rightarrow$ (2x) Velocity constraints (i.e. $v_g \cdot \hat{n}_i = 0$)

Since there are two non-conservative forces acting on the cart, we must modify the Lagrange equation such that we account for the lack of gravity and generated forces on the system.

$$\mathcal{L} = E_k - E_p \tag{1}$$

$$E_p = 0 \tag{2}$$

$$E_k = \frac{1}{2}(\dot{x}_G^2 + \dot{y}_G^2 + I\dot{\theta}^2) \tag{3}$$

After identifying the necessary variables to describe our motion, can describe our inertial equations in the state

$$\begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix} = \begin{bmatrix} x_G \\ y_G \\ \theta \end{bmatrix}$$

Adding in the generalized forces into our Lagrange equation

$$\frac{d}{dt}\frac{\partial \mathcal{L}}{d\dot{q}_i} - \frac{\partial \mathcal{L}}{\partial \dot{q}_i} = Q_i \tag{4}$$

where

$$Q_i = \sum \vec{F}_i \cdot \frac{\partial \vec{r}_i}{\partial q_i} \tag{5}$$

$$Q_x = \frac{\partial \vec{r}_1}{\partial x_G} \cdot N_1\hat{n}_1 + \frac{\partial \vec{r}_2}{\partial x_G} \cdot N_2\hat{n}_2 = \hat{i} \cdot N_1\hat{n}_1 + \hat{i} \cdot N_1\hat{n}_2 \tag{6}$$

$$Q_y = \frac{\partial \vec{r}_1}{\partial y_G} \cdot N_1\hat{n}_1 + \frac{\partial \vec{r}_2}{\partial y_G} \cdot N_2\hat{n}_2 = \hat{j} \cdot N_1\hat{n}_1 + \hat{j} \cdot N_1\hat{n}_2 \tag{7}$$

$$Q_\theta = \frac{\partial \vec{r}_1}{\partial \theta} \cdot N_1\hat{n}_1 + \frac{\partial \vec{r}_2}{\partial \theta} \cdot N_2\hat{n}_2 = \vec{J}_1 \cdot N_1\hat{n}_1 + \vec{J}_2 \cdot N_1\hat{n}_2 \tag{8}$$

J_1 and J_2 are Jacobians describing the rotational motion a

Using the Lagrangian to solve for Q_x, Q_y and Q_θ, we find that

$$\begin{bmatrix} Q_x \\ Q_y \\ Q_\theta \end{bmatrix} = \begin{bmatrix} m\ddot{x}_G \\ m\ddot{y}_G \\ I\ddot{\theta} \end{bmatrix}$$

Therefore our equations of motion are

$$\begin{bmatrix} m\ddot{x}_G \\ m\ddot{y}_G \\ I\ddot{\theta} \end{bmatrix} = \begin{bmatrix} \hat{i} \cdot N_1\hat{n}_1 + \hat{i} \cdot N_1\hat{n}_2 \\ \hat{j} \cdot N_1\hat{n}_1 + \hat{j} \cdot N_1\hat{n}_2 \\ \vec{J}_1 \cdot N_1\hat{n}_1 + \vec{J}_2 \cdot N_1\hat{n}_2 \end{bmatrix}$$

It is important to note that we should leave these all in terms of the unit vectors so we can simply put them into MatLAB. There is no sense in transforming these equations into $\hat{i}$ and $\hat{j}$.

Now that we have the equations of motion, we must solve for the constraint equations. We will twice differentiate the position vectors that describe the location of each skate in frame F. To do this, we will use $\vec{v}_1 \cdot \hat{n}_1 = 0$ and $\vec{v}_2 \cdot \hat{n}_2 = 0$ (with the subscript 1 denoting the first skate and subscript 2 the second skate). Then we will differentiate that result to solve for $a_1 = 0$ and $a_2 = 0$.

$$\begin{aligned} \vec{r}_1 &= \vec{r}_G + \vec{r}_{1/G} \\ &= x_G\hat{i} + y_G\hat{j} - d\hat{\lambda}_1 \end{aligned} \tag{9}$$

$$\begin{aligned} \vec{r}_2 &= \vec{r}_G + \vec{r}_{2/G} \\ &= x_G\hat{i} + y_G\hat{j} + d\hat{\lambda}_1 \end{aligned} \tag{10}$$

Solve for the velocity vectors and dot them with their frame's respective $\hat{n}$ unit vectors.

$$\vec{v}_1 \cdot \hat{n}_1 = 0 \tag{11}$$

$$(\dot{x}_G\hat{i} + \dot{y}_G\hat{j} - d\dot{\theta}\hat{n}_1) \cdot \hat{n}_1 = 0 \tag{12}$$

$$\vec{v}_2 \cdot \hat{n}_2 = 0 \tag{13}$$

$$(\dot{x}_G\hat{i} + \dot{y}_G\hat{j} + d\dot{\theta}\hat{n}_1) \cdot \hat{n}_2 = 0 \tag{14}$$

Now that we have the velocities (scalar!!!) for each skate, we can solve for the accelerations (also scalar!!!) for our constraints. Note that in the following equations we will include our unit vectors so our calculation in MatLAB will be much more straightforward.

$$a_1 = (\ddot{x}_G\hat{i} + \ddot{y}_G\hat{j} + d\dot{\theta}^2\hat{n}_1 - d\ddot{\theta}\hat{\lambda}_1) \cdot \hat{n}_1 + (\dot{x}_G\hat{i} + \dot{y}_G\hat{j} - d\dot{\theta}\hat{n}_1) \cdot -\dot{\theta}\hat{\lambda}_1 \tag{15}$$

$$a_2 = (\ddot{x}_G\hat{i} + \ddot{y}_G\hat{j} - d\dot{\theta}^2\hat{n}_1 + d\ddot{\theta}\hat{\lambda}_1) \cdot \hat{n}_2 + (\dot{x}_G\hat{i} + \dot{y}_G\hat{j} + d\dot{\theta}\hat{n}_1) \cdot -\dot{\theta}\hat{\lambda}_2 \tag{16}$$

Now that we have our 3 equations of motion and 2 constraint equations, we can solve for the DAEs in Matlab using the $A\backslash b$ command. In summary, our DAEs are

$$\begin{bmatrix} m\ddot{x}_G \\ m\ddot{y}_G \\ I\ddot{\theta} \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} \hat{i} \cdot N_1\hat{n}_1 + \hat{i} \cdot N_1\hat{n}_2 \\ \hat{j} \cdot N_1\hat{n}_1 + \hat{j} \cdot N_1\hat{n}_2 \\ \vec{J}_1 \cdot N_1\hat{n}_1 + \vec{J}_2 \cdot N_1\hat{n}_2 \\ 0 \\ 0 \end{bmatrix}$$

Now that we have our DAEs, it is time to put everything into MatLAB. Solving for our twice-differentiated equations will require a "driving" script that calls ode45(@RHS) and a "deriver" function that will help solve for our first-order matrix by returning our RHS function. The deriver will also give us our worked-out velocity equations so we can solve for the non-trivial initial conditions.

We will first solve for the "derive" function that will give us our RHS function.

The Deriver Function

```
function Problem34Derive()
clear all; close all;

%% Create the symbols necessary for
%Essential variables:
  syms th thdot thddot phi real
  syms xg yg xgdot ygdot xgddot ygddot real
%Intermediate variables:
  syms lam1 lam2 n1 n2 iHat jHat kHat A B C rG rA rB rArelG rBrelG real
  syms a1 a2 v1 v2 J1 J2 N1 N2 real
%Constant parameters:
  syms m d I zddray real

%% Establish our unit vectors for each frame
% Inertial unit vectors
iHat    = [1          0         0];
jHat    = [0          1         0];
kHat    = [0          0         1];
% Rotating Frame for A
lam1 = [cos(th)   sin(th)  0];
n1   = [-sin(th)  cos(th)  0];
A    = [lam1; n1; kHat];
% Rotation matrix for B
lam2 = [cos(phi)   sin(phi)  0];
n2   = [-sin(phi)  cos(phi)  0];
B    = [lam2; n2; kHat];
% Put our rotation matrix in terms of inertial unit vectors
C    = A*B;
% Extract the unit vectors of interest and rewrite variables lam2 and n2
lam2 = C(1,:);
n2   = C(2,:);

%% Solve for the equations of motion in terms of Qx, Qy, Qth
%%%%% NOTE: in this code, to reduce some confusion, the orgin of the frame
%%%%% A refers to the origin of frame B in the diagram and the origin of
%%%%% the frame B refers to the origin of the frame gamma. The unit vectors
%%%%% are true for both the code and the diagram
rG = xg*iHat + yg*jHat;
rArelG = -d*lam1;
rBrelG = d*lam1;
rA = rG + rArelG;
rB = rG + rBrelG;

% Input our general forces Qx and Qy
Qx = N1*iHat*n1'  + N2*iHat*n2';
Qy = N1*jHat*n1'  + N2*jHat*n2';
% Simplify these equations
Qx = simplify(Qx);
Qy = simplify(Qy);

% Solve for the Qtheta general force. For this equation, we must use the
% jacobian command to derive each position vector with respect to the
% interial rotation variable, theta.
J1 = jacobian(rA,th);
J2 = jacobian(rB,th);
Qth= N1*n1*J1 + N2*n2*J2;
% Simplify the equation
Qth= simplify(Qth);

%% Solve for the constraints
%%% NOTE: we don't necessarily have to solve for the velocity equations
```

```
%%% here but I did just to check my progress and ensure that I will derive
%%% my acceleration terms, the constraints, properly. These will also help
%%% us to solve for the system's initial conditions.

% Solve for our each skate's velocity by dotting with normal unit vector
v1 = (xgdot*iHat + ygdot+jHat - d*thdot*n1)*n1';
v2 = (xgdot*iHat + ygdot*jHat + d*thdot*n1)*n2';
% Simplify the equations and check!
v1s = simplify(v1);
v2s = simplify(v2);

% Solve for our constraints!!!
a1 = (xgddot*iHat + ygddot*jHat - d*thddot*n1 + d*thdot^2*lam1)*n1'...
    + (xgdot*iHat + ygdot*jHat - d*thdot*n1)*-thdot*lam1';
a2 = (xgddot*iHat + ygddot*jHat + d*thddot*n1 - d*thdot^2*lam1)*n2'...
    + (xgdot*iHat + ygdot*jHat + d*thdot*n1)*-thdot*lam2';

% Simplify
a1s= simplify(a1);
a2s= simplify(a2);

% We will use the equationsToMatrix(eqns,vec) function to solve for our A
% and b matrices. First we must create a [eqns] vector and [vars] state vec.
eqns = [m*xgddot == Qx
        m*ygddot == Qy
        I*thddot == Qth
        a1s == 0
        a2s == 0];
var = [xgddot
       ygddot
       thddot
       N1
       N2];
% Solve for the A and b matrices
[A,b] = equationsToMatrix(eqns,var);

% MAGIC!!! We will put our second-order diff-eqns into a temporary array of
% size 5x1
zddray = A\b;

% Extract the the second order equations from the temporary array, zddray,
% and convert them into type char. This will allow us to assign the
% variables a value in our driving script
xgddot = char(zddray(1));
ygddot = char(zddray(2));
thddot = char(zddray(3));

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Put derived equations  in a right-hand-side file ode45.

fid=fopen(   'Problem34rhs.m','w'                                          );
fprintf(fid, 'function zdot=Problem34rhs(t,z,flag,m,I,d,phi)\n'       );
fprintf(fid, 'xg     = z(1);                                          \n');
fprintf(fid, 'xgdot  = z(2);                                          \n');
fprintf(fid, 'yg     = z(3);                                          \n');
fprintf(fid, 'ygdot  = z(4);                                          \n');
fprintf(fid, 'th     = z(5);                                          \n');
fprintf(fid, 'thdot  = z(6);                                          \n');

fprintf(fid,['xgddot = ' xgddot ';                                 \n\n']);
fprintf(fid,['ygddot = ' ygddot ';                                 \n\n']);
fprintf(fid,['thddot = ' thddot ';                                 \n\n']);
% Create the return vector zdot for our RHS function.
fprintf(fid, 'zdot = [xgdot xgddot ygdot ygddot thdot thddot]'';         \n');
fprintf(fid, 'end                                                     \n');
fclose(fid);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

end
```

Solving for the Initial Conditions

When solving for the initial conditions, we will set some "known" parameters $\phi = \frac{\pi}{4}$, $d = 1$, and $\theta = 0$. We will also set the speed of the cart in the x-direction, $\dot{x}_G$, to arbitrarily be 1.

$$\begin{aligned} v_1 &= -\dot{x}_G sin(\theta) + \dot{y}_G cos(\theta) - d\dot{\theta} = 0 \\ v_2 &= -\dot{x}_G cos(\phi + \theta) + \dot{y}_G cos(\phi + \theta) + d\dot{\theta} = 0 \end{aligned} \tag{17}$$

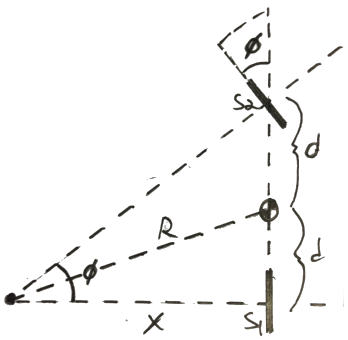
Substituting in our known variables, we find

$$\begin{aligned} v_1 &\Rightarrow \dot{y}_G = \dot{\theta} \\ v_2 &\Rightarrow \dot{y}_G + \dot{\theta} = 1 \end{aligned} \tag{18}$$

Further algebra and substitution reveals our initial conditions:

$$\begin{bmatrix} x_G \\ \dot{x}_G \\ y_G \\ \dot{y}_G \\ \theta \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1/2 \\ 0 \\ 1/2 \end{bmatrix}$$

The Simple Solution



We can draw out a diagram for the simple solution using basic trigonometry. It is important to note the relationship between the angle, ϕ of the second skate and the angle of the system. This "steering" angle will define the radius of the cart's turn. Using the above relation, we can work out the governing equations of motion.

$$x = \frac{2d}{tan(\phi)} \tag{19}$$

$$R = \sqrt{d^2 + x^2} \tag{20}$$

$$R = \sqrt{d^2 + \frac{2d}{tan(\phi)}} \tag{21}$$

When solving in MatLAB, the cartesian coordinates are solved for by the polar coordinate transformation $x = Rcos(\theta)$ and $y = Rsin(\theta)$, where θ is the angle of the cart with respect to the horizontal $(\hat{i})$.

MatLAB Driving Function

Now that we have a set of initial conditions and the simple solution, we can write the script for the driving function and compare our Lagrange solution to our "simple solution".

```
% Problem 34 Driver
clear all; close all;

%% Basic Parameters
m = 1;
d = 1;
I = 1;
phi = pi/4;

%% Call the deriver function. This will generate our RHS function
Problem34Derive

%% Initial Conditions
xINT  = 0;
yINT  = 0;
thINT = 0;
xdINT = 1;
ydINT = 1/2;
thdINT= 1/2;
% Pack them into a single vector
z0 = [xINT xdINT yINT ydINT thINT thdINT];

% Time for ode45 solving function
tspan = 0:.01:20;

% Set integration tolerances. Use default here
opts.RelTol = 1e-6;
opts.AbsTol = 1e-6;

%% Call the ode45 function
[tarray,zarray] = ode45(@Problem34rhs,tspan,z0,opts,flag,m,I,d,phi);

% Unpack the results into usable variables.
x = zarray(:,1);
y = zarray(:,3);
th= zarray(:,5);

%% Simple Solution
xAna = 2*d/tan(phi);
R = sqrt(d^2+xAna^2);
xSimp = -1+R*cos(th-45);   % Shift graph to account for the initial conditions
ySimp =  2+R*sin(th-45);   % Shift graph to account for the initial conditions

%% Plot the results
figure
hold on
box on
axis square
title('Two Skates on a Plane','Fontsize',14)
xlabel('X','Fontsize',14)
ylabel('Y','Fontsize',14)
% Lagrange solution
plot(x,y,'Linewidth',5)
% Simple solution
plot(xSimp,ySimp,'Linewidth',2)
legend({'Lagrange','Simple'},'Fontsize',12)
```

Lagrange vs. Simple Plot Comparison